

Unit 4: Data Preprocessing

Data Cleaning Techniques – Class Activity Sheet #6

Mathematical methods and Pandas code for missing values and outliers

Name		Batch	
Date		Global ID	

Learning Objectives

- Identify missing values, duplicates, inconsistent labels, wrong data types, and outliers.
- Choose suitable cleaning actions for simple datasets.
- Apply basic data cleaning commands using Pandas.

Part A: Concept Recall

Data cleaning is the process of finding and correcting problems in a dataset before analysis. Poor quality data can lead to wrong conclusions, weak models, and misleading reports.

Data Issue	Simple Meaning	Example
Missing value	A value is blank or not available	Age is empty
Duplicate row	The same record appears more than once	Same student entered twice
Inconsistent text	Same category written in different ways	Male, male, M
Wrong data type	A column has an unsuitable format	Age stored as text
Outlier	A value is unusually high or low	Age = 250

Part B: Identify the Data Cleaning Problem

S. No.	Data Example	Problem Type
1	Marks column contains blank cells	
2	Gender contains Male, male, M, Female, F	
3	Two rows have the same Student ID	
4	Age column contains 230	
5	Date is stored as 01/02/2026 in one row and 2026-02-01 in another row	
6	Salary column contains Rs. 50,000 as text	

Part C: Dataset 1 - Student Records

Student_ID	Name	Age	Gender	Marks	City
101	Anu	19	Female	78	Delhi
102	Ravi		male	85	dehradun
103	Meena	20	F		Delhi
103	Meena	20	F		Delhi
104	Karan	230	M	64	Mumbai
105	Sara	21	Female	91	mumbai

Task: Identify all data quality issues and suggest cleaning actions.

1. Which columns contain missing values?
2. Which row is duplicated?
3. Which column has inconsistent category labels?
4. Which value appears to be an outlier?
5. How will you clean the City column?

Part D: Dataset 2 - Sales Records

Order_ID	Product	Quantity	Price	Date	Payment_Mode
501	Laptop	1	55000	01-08-2026	UPI
502	Mouse	2	Rs. 500	02-08-2026	upi
503	Keyboard		1200	03-08-2026	Cash
504	Monitor	1	-15000	04-08-2026	CARD
505	Mouse	2	500	02-08-2026	UPI

6. Which column has inconsistent text values?
7. Which column has a missing value?
8. Which value is invalid in the Price column?
9. Which column needs date format standardization?
10. Write one cleaning rule for the Payment_Mode column.

Part E: Missing Value Techniques

Missing values can be handled by deletion, mean imputation, median imputation, mode imputation, forward fill, backward fill, or interpolation. The method should be selected based on the type of data.

Technique	Mathematical Idea	Best Used For
Deletion	Remove rows where x is missing	Very few missing rows
Mean imputation	$x_{\text{missing}} = \text{mean}(x)$	Numerical data without strong outliers
Median imputation	$x_{\text{missing}} = \text{median}(x)$	Numerical data with outliers
Mode imputation	$x_{\text{missing}} = \text{mode}(x)$	Categorical data
Forward fill	$x_t = x_{(t-1)}$	Time series data
Linear interpolation	$x_t = x_1 + ((t - t_1)/(t_2 - t_1))(x_2 - x_1)$	Ordered numerical data

Activity E1: Mean, Median, and Mode Imputation

Dataset: Marks = 40, 50, missing, 70, 80

Mean formula: mean = sum of observed values / number of observed values

Mean = $(40 + 50 + 70 + 80) / 4 = \underline{\hspace{2cm}}$

Replace missing value using mean: $\underline{\hspace{2cm}}$

Now try median imputation and write the replaced value: $\underline{\hspace{2cm}}$

For categorical data, use mode. Example: City = Delhi, Delhi, Mumbai, missing. Mode = $\underline{\hspace{2cm}}$

Python code:

```
import pandas as pd
df = pd.DataFrame({
    "Marks": [40, 50, None, 70, 80],
    "City": ["Delhi", "Delhi", "Mumbai", None, "Delhi"]
})
df["Marks_mean"] = df["Marks"].fillna(df["Marks"].mean())
df["Marks_median"] = df["Marks"].fillna(df["Marks"].median())
df["City_mode"] = df["City"].fillna(df["City"].mode()[0])
print(df)
```

Activity E2: Forward Fill and Interpolation

Day	Temperature
Day 1	30
Day 2	
Day 3	34
Day 4	
Day 5	38

11. Use forward fill to replace the missing value on Day 2.
12. Use linear interpolation to replace the missing value on Day 2.
13. Which method is better when the data follows a gradual trend?

Python code:

```
df = pd.DataFrame({"Temperature": [30, None, 34, None, 38]})
df["Forward_Fill"] = df["Temperature"].ffill()
df["Backward_Fill"] = df["Temperature"].bfill()
df["Interpolated"] = df["Temperature"].interpolate()
print(df)
```

Part F: Outlier Detection Techniques

An outlier is a value that is unusually far from the other values. Outliers can be detected using domain rules, Z-score, IQR, boxplots, or percentile limits.

Technique	Formula / Rule	Decision
Domain rule	Use subject knowledge	Age > 100 may be invalid
Z-score	$z = (x - \text{mean}) / \text{standard deviation}$	If $ z > 3$, mark as an outlier
IQR method	$IQR = Q3 - Q1$	Outlier if $x < Q1 - 1.5IQR$ or $x > Q3 + 1.5IQR$
Percentile capping	Cap below P1 and above p99	Limits extreme values

Activity F1: Z-score Method

Dataset: Salary = 20, 22, 23, 24, 25, 200

Step 1: Find mean. Step 2: Find standard deviation. Step 3: Calculate z for each value.

Formula: $z = (x - \text{mean}) / \text{standard deviation}$

If absolute z-score is greater than 3, the value is treated as an outlier.

Python code:

```
import pandas as pd
df = pd.DataFrame({"Salary": [20, 22, 23, 24, 25, 200]})
mean_salary = df["Salary"].mean()
std_salary = df["Salary"].std(ddof=1)
df["z_score"] = (df["Salary"] - mean_salary) / std_salary
df["Outlier"] = df["z_score"].abs() > 3
print(df)
```

Activity F2: IQR Method

Dataset: Price = 10, 12, 13, 14, 15, 100

Q1 = 25th percentile, Q3 = 75th percentile, IQR = Q3 - Q1

Lower limit = Q1 - 1.5 x IQR

Upper limit = Q3 + 1.5 x IQR

Values outside these limits are outliers.

Python code:

```
df = pd.DataFrame({"Price": [10, 12, 13, 14, 15, 100]})
Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df["Outlier"] = (df["Price"] < lower) | (df["Price"] > upper)
print("Q1:", Q1, "Q3:", Q3, "IQR:", IQR)
print("Lower:", lower, "Upper:", upper)
print(df)
```

Part G: Outlier Treatment Techniques

Treatment	Mathematical / Practical Idea	Code Function
Remove	Drop rows containing outliers	filter condition
Cap	Values below lower become lower; values above upper become upper	clip(lower, upper)
Transform	Use $\log(x)$ to reduce skewness	$\text{np.log1p}(x)$
Replace	Replace invalid outlier with median	fillna(median)

Python code:

```
import numpy as np
df = pd.DataFrame({"Price": [10, 12, 13, 14, 15, 100]})
Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# 1. Remove outliers
df_removed = df[(df["Price"] >= lower) & (df["Price"] <= upper)]
# 2. Cap outliers
df["Price_capped"] = df["Price"].clip(lower, upper)
# 3. Log transformation
df["Price_log"] = np.log1p(df["Price"])
print(df_removed)
print(df)
```

Part I: Cleaning Decision Table

Complete the table by writing the best cleaning action.

Issue Found	Cleaning Action
Missing numerical value	
Missing categorical value	
Duplicate row	
Gender written as M, Male, male	
City written as Delhi, delhi, DELHI	
Age value is 230	
Price contains Rs. symbol	

Part J: Before and After Cleaning

Observe the dirty data and write the cleaned version.

Column	Dirty Value	Cleaned Value
Gender	male	
Gender	F	
City	mumbai	
Payment_Mode	upi	
Price	Rs. 500	
Date	2026/08/02	

Part K: Combined Python Practice

Use the following Pandas code to detect and clean common data issues.

```
import pandas as pd

data = {
    "Student_ID": [101, 102, 103, 103, 104, 105],
    "Name": ["Anu", "Ravi", "Meena", "Meena", "Karan", "Sara"],
    "Age": [19, None, 20, 20, 230, 21],
    "Gender": ["Female", "male", "F", "F", "M", "Female"],
    "Marks": [78, 85, None, None, 64, 91],
    "City": ["Delhi", "dehradun", "Delhi", "Delhi", "Mumbai", "mumbai"]
}

df = pd.DataFrame(data)

print(df.isnull().sum())
print(df.duplicated().sum())

df = df.drop_duplicates()
df["Gender"] = df["Gender"].replace({"male": "Male", "M": "Male", "F": "Female"})
df["City"] = df["City"].str.title()
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df.loc[df["Age"] > 100, "Age"] = None
print(df)
```

14. What does `isnull().sum()` show?
15. What does `duplicated().sum()` show?
16. Why is `drop_duplicates()` used?
17. Why do we replace M and F with Male and Female?
18. Why is Age greater than 100 treated as an invalid value?

Answer Key

Part B: 1. Missing value, 2. Inconsistent text, 3. Duplicate row, 4. Outlier, 5. Inconsistent date format, 6. Wrong data type or unwanted symbol.

Dataset 1: Missing values are Age and Marks. Student_ID 103 is duplicated. Gender and City need standardization. Age 230 is an outlier or invalid value.

Dataset 2: Payment_Mode and Date need standardization. Quantity has a missing value. Price - 15000 is invalid because price should not be negative. Rs. 500 should be converted to a number.

Conclusion example: After cleaning, the dataset becomes more consistent, reliable, and suitable for analysis.